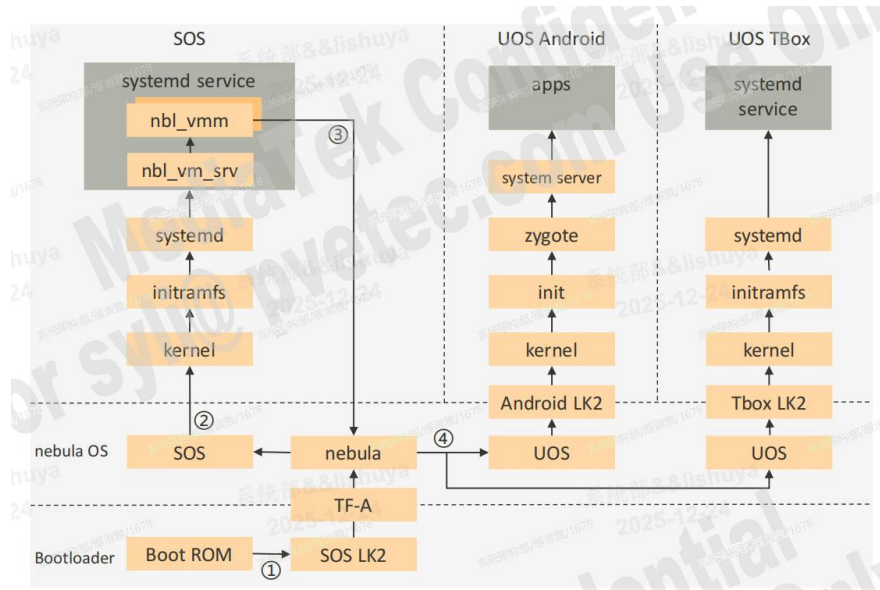


一、Hypervisor Bootup 流程



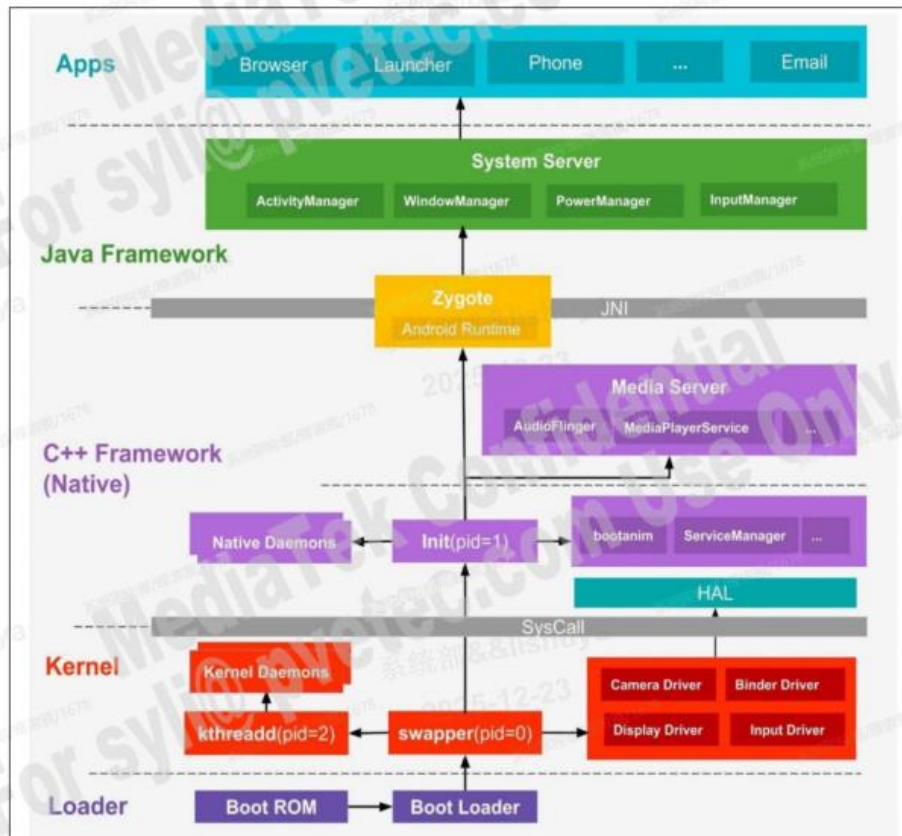
1、当平台上电之后，Boot ROM 会加载并跳转到 SOS LK2。SOS LK2 会完成一些基本的硬件初始化，加载 kernel、tinysys、tee、gz 等镜像，最后通过 TF-A 跳转到 nebula，开始初始化虚拟机。虚拟机在完成自身的一些初始化之后，会启动一个名为 SOS 的任务，根据相关配置文件，启动 SOS 虚拟机。

2、SOS 虚拟机按照 kernel、initramfs、systemd 的顺序依次完成初始化。systemd 初始化完成之后，会启动各种用户层服务，其中一个虚拟机服务进程 nbl_vm_srv。nbl_vm_srv 会解析配置文件 vm_srv_cfg.pb.txt，根据相应的配置启动 UOS 虚拟机。

3、对于每一个 UOS 虚拟机，nbl_vm_srv 进程会启动一个对应的虚拟机管理进程 nbl_vmm。nbl_vmm 进程会读取虚拟机配置文件 uos_alps_pv8676.lua（或 uos_tbox_pv8676.lua），根据配置创建 bootloader、初始化后端驱动，并序列化 CPU、内存、irq 等配置，传递给 nebula 启动 UOS。

4、nebula 创建好 UOS 虚拟机后，跳转到 UOS LK2，开始启动 UOS。最后，UOS 开始自己的启动流程。SOS 启动完成之后，服务 nbl_yocto_cpu_offline.service 会按 CPU 分配策略关闭 SOS 的 vCPU。UOS Android 启动完成后，init.project.rc 会按 CPU 分配策略关闭 UOS Android 的 vCPU。UOS TBox 启动完成后，会在 initramfs 阶段的 uos_earlyinit 脚本、systemd 阶段的 uos_init.service 按 CPU 分配策略关闭 UOS TBox 的 CPU。

二、android 端 Bootup 流程



- 1、loader 层：上电之后，先执行 bootrom 中的 bootcode，将 bootloader 从外部 flash 加载到 SRAM 中，对 DRAM 进行初始化，拉起 Linux kernel。Boot Loader 分为 preloader 和 LK 两大部分，preloader 运行在 SDRAM 中，LK 运行在 DRAM 中，LK 会去加载 kernel 程序。
- 2、kernel 层：首先启动 0 号进程，用于初始化进程管理、内存管理、加载驱动程序；再 fork 出 2 号进程，2 号进程是所有 Linux 内核进程的鼻祖。
- 3、native 层：kernel 层的驱动程序加载完毕之后，硬件设备驱动与 HAL 层进行交互，启动 1 号进程。
- 4、native 层：init 进程（1 号进程）启动之后会启动各种用户守护进程，并且会启动 servicemanager（binder 服务管家）等重要服务，同时解析对应的配置文件 init.rc 并且 fork 出 Zygote 进程。
- 5、Zygote 进程是介于 native 层与 Java 层的进程，它会加载虚拟机 JVM，注册 JNI 函数，打通 native 到 Java 层的通道，并且启动 systemserver（负责整个 Java 框架）以及第一个应用程序进程启动器。

三、Android Bootup 日志分析

- 1、系统起不来问题首先需要抓串口 log，根据需要也可能要抓 logcat

- (1) 抓份不能开机的串口 log
- (2) 抓份正常开机的串口 log 用来对比

- 2、检查关键字或者关键 log

```
[ 28.914299][T1200903] kworker/u16:3: [name:nebula_log_driver&]nebula: (4)[00036.748]
02443.03973> [INFO:vhm_service_impl.cc(162)] VM - 0 Created // 创建 android 虚拟机
```

```

...
[ 29.190462][T1400010] kworker/u16:1: [name:nebula_log_driver&]nebula: (4)[00036.980]
02443.03973> [INFO:vhm_service_impl.cc(232)] VM - 0 Start Success // 开始加载 android 虚拟机
...
rst from: kernel //Android 系统复位时的状态和寄存器值，可判断上次在什么阶段重启
MODE:          0x14d
STA:           0x0
LENGTH:       0x16800
INTERVAL:      0xffff
SWSYSRST:      0x0
LATCH_CTL:     0xf21e79
NONRST_REG:    0x0
NONRST_REG2:   0x6c000800
DEBUG_CTL:     0x0
REQ_MODE:      0xfd00e3
REQ_IRQ_EN:    0x7d00e1
...
welcome to lk2 // android lk2 start
lk variant: BL33
boot args 0x0 0x0 0x0 0x0
version:
    arch:      arm64
    platform:  mediatek
    target:     auto8676p1_64_ufs
    project:    auto8676p1_64_hyp-an
    buildid:    202512181841_
hello platform_init
...
boot_linux_fdt:728: lk finished --> jump to linux kernel 64Bit // android LK2 jumps to android
kernel
...
[ 30.719316][T1300010] kworker/u16:1: [name:nebula_log_driver&]nebula: (2)[00038.555]
04922.05461> [ 0.000000][ T0] swapper: [name:setup&]Booting Linux on physical CPU
0x000000000000 [0x411fd461] // start android kernel
[ 30.724035][T1300010] kworker/u16:1: [name:nebula_log_driver&]nebula: (2)[00038.557]
04922.05461> [ 0.000000][ T0] swapper: [name:main&]Linux version 6.1.124
(oe-user@oe-host) (clang version 18.1.2 (https://github.com/llvm/llvm-project
26a1d6601d727a96f4301d0d8647b5a42760ae0c), GNU ld (GNU Binutils) 2.42.0.20240216
[ 30.731327][T1400010] kworker/u16:1: [name:nebula_log_driver&]nebula: (2)[00038.559]
04922.05461> ) #1 SMP Wed May 28 06:47:14 UTC 2025
[ 30.732861][T1400010] kworker/u16:1: [name:nebula_log_driver&]nebula: (2)[00038.562]
04922.05461> [ 0.000000][ T0] swapper: [name:setup&]Machine model: MT8646
[ 30.734718][T1400010] kworker/u16:1: [name:nebula_log_driver&]nebula: (2)[00038.563]

```

```
04922.05461> [ 0.000000][ T0] swapper: earlycon: [name:earlycon&]pl11 at MMIO
0x0000000ffff32000 (options "")
[ 30.734720][T1400010] kworker/u16:1: [name:nebula_log_driver&]nebula: (2)[00038.565]
04922.05461> [ 0.000000][ T0] swapper: printk: bootconsole [pl11] enabled
...
[ 34.071500][T1700010] kworker/u16:1: [name:nebula_log_driver&]nebula: (5)[00041.519]
04125.05839> [ 2.327425][T500001] init: init 3: init first stage started!
[ 34.071501][T1700010] kworker/u16:1: [name:nebula_log_driver&]nebula: (5)[00041.521]
04125.05839> [ 2.330532][T500001] init: init 3: Loading module
/lib/modules/virtio_mmio.ko with args " // android init 加载 ko 阶段
...
[ 35.209199] nbl_vm_srv[2580]: [vmid:0][ 3.660073][T500001] init:
[name:bootprof&]BOOTPROF: 3660.072931:Kernel_init_done // android kernel init done
...
[ 45.693227] nbl_vm_srv[2580]: [vmid:0][ 14.300423][T600001] init: init 3: init second
stage started!
...
[ 47.177577] nbl_vm_srv[2580]: [vmid:0][ 15.776582][T600001] init: init 22: Parsing file
/system/etc/init/hw/init.rc... //android init 解析 rc 文件
[ 47.177686] nbl_vm_srv[2580]: [vmid:0][ 15.780169][T600001] init: init 22: Added
'/init.envIRON.rc' to import list
[ 47.177780] nbl_vm_srv[2580]: [vmid:0][ 15.782095][T600001] init: init 22: Added
'/system/etc/init/hw/init.usb.rc' to import list
[ 47.177891] nbl_vm_srv[2580]: [vmid:0][ 15.786321][T600001] init: init 22: Added
'/init.mt8676.rc' to import list
[ 47.178364] nbl_vm_srv[2580]: [vmid:0][ 15.790095][T600001] init: init 22: Added
'/vendor/etc/init/hw/init.mt8676.rc' to import list
[ 47.178461] nbl_vm_srv[2580]: [vmid:0][ 15.793129][T600001] init: init 22: Added
'/system/etc/init/hw/init.usb.configfs.rc' to import list
...
[ 60.254774] nbl_vm_srv[2580]: [vmid:0][ 28.655972][T400282] init:
[name:bootprof&]BOOTPROF:modprobe: Load_Module_DONE // android ko 加载完成
...
[ 61.429593] nbl_vm_srv[2580]: [vmid:0][ 30.083792][T500001] init: init 37:
[29985][0]processing action (zygote-start) from (/vendor/etc/init/hw/init.mt8676.rc:757)
[ 61.429717] nbl_vm_srv[2580]: [vmid:0][ 30.090718][T600282] init:
[name:bootprof&]BOOTPROF: 30090.716917:INIT:zygote-start // 启动 android zygote 进程
...
[name:bootprof&]BOOTPROF: Zygote:Preload End
...
[ 61.516762] nbl_vm_srv[2580]: [vmid:0][ 30.108108][T500282] init:
[name:bootprof&]BOOTPROF: 30108.106763:INIT:early-boot // android 进入 early-boot 阶
段，负责一些基础服务和参数的设置
...
```

```
[ 68.947613] nbl_vm_srv[2580]: [vmid:0][ 36.379110][T601083] system_server:
[name:bootprof&]BOOTPROF: 36379.108394:Android:SysServerInit_START // 启动 system
server
...
[ 77.160856] nbl_vm_srv[2580]: [vmid:0][ 45.560415][T201083] system_server:
[name:bootprof&]BOOTPROF: 45560.411492:Android:SysServerInit_END
...
[ 79.435652] nbl_vm_srv[2580]: [vmid:0][ 48.057200][T500282] init:
[name:bootprof&]BOOTPROF: 48057.199422: OFF (KO:236) // android 开机完成
```

(1) 如果串口 log 里面没有走到 LK, 说明在 PL 就卡住了

(2) "lk finished --> jump to linux kernel"

如果串口 log 里面没有走到这里, 但是上面走到了, 说明在 lk 卡住了, 这句表示进入 kernel

(3) kernel 日志获取

① 通过串口获取, 过滤 vmid:0

② 导出 yocto 和 android 端的 debuglogger

(4) "Kernel_init_done"

这句表示 kernel 初始化完成了, 正准备启动 init 进程, 如果这没有走到: 请搜索关键字 initcall, initcall 是顺序执行例如 module_init 注册的 init 函数, 看看卡在哪个 call back 上

(5) "BOOTPROF"

这是开机过程中, 每走过一个重要阶段都会打印的标志, 例如上面 Kernel_init_done 这一句就会打上这个标志

(6) "init first stage"

这句表示 init 进程正常起来执行

Init 正常启动

init: init 3: init first stage started!

init: init 3: init second stage started!

mount 成功

```
[ 44.419782] nbl_vm_srv[2580]: [vmid:0][ 12.763235][T600001] init: init 6: [libfs_mgr]
__mount(source=/dev/block/by-name/metadata,target=/metadata,type=f2fs)=0: Success
[ 44.694849] nbl_vm_srv[2580]: [vmid:0][ 12.967287][T500001] init: init 6: [libfs_mgr]
__mount(source=/dev/block/dm-8,target=/system,type=erofs)=0: Success
[ 44.822179] nbl_vm_srv[2580]: [vmid:0][ 13.019785][T500001] init: init 9: [libfs_mgr]
__mount(source=/dev/block/dm-9,target=/system_ext,type=erofs)=0: Success
[ 44.824283] nbl_vm_srv[2580]: [vmid:0][ 13.058401][T500001] init: init 6: [libfs_mgr]
__mount(source=/dev/block/dm-10,target=/vendor,type=erofs)=0: Success
[ 44.954297] nbl_vm_srv[2580]: [vmid:0][ 13.089545][T500001] init: init 6: [libfs_mgr]
__mount(source=/dev/block/dm-11,target=/product,type=erofs)=0: Success
[ 44.955448] nbl_vm_srv[2580]: [vmid:0][ 13.124761][T500001] init: init 6: [libfs_mgr]
__mount(source=/dev/block/dm-12,target=/vendor_dlkm,type=erofs)=0: Success
[ 45.071713] nbl_vm_srv[2580]: [vmid:0][ 13.155828][T500001] init: init 9: [libfs_mgr]
__mount(source=/dev/block/dm-13,target=/odm_dlkm,type=erofs)=0: Success
[ 45.192084] nbl_vm_srv[2580]: [vmid:0][ 13.185361][T500001] init: init 9: [libfs_mgr]
__mount(source=/dev/block/dm-14,target=/system_dlkm,type=erofs)=0: Success
```

```
[ 50.374773] nbl_vm_srv[2580]: [vmid:0][ 18.683285][T500001] init: init 25:
[18196][0][libfs_mgr]__mount(source=/dev/block/by-name/nvcfg,target=/mnt/vendor/nvcfg,type
e=ext4)=0: Success
```

异常日志示例如下

```
init: init 24: [33136][0][libfs_mgr]__mount(source=/dev/block/by-name/userdata,target=/data,type=f2fs)=1: Device or resource busy
init: init 24: [33136][0][libfs_mgr]Running /system/bin/fsck.f2fs -f -c 10000 --debug-cache /dev/block/vda87
init: init 24: [33136][0][libfs_mgr] /system/bin/fsck.f2fs returned status 0x800
init: init 24: [33136][0][libfs_mgr]__mount(source=/dev/block/by-name/userdata,target=/data,type=f2fs)=1: Device or resource busy
init: init 25: [33136][0][libfs_mgr]Failed to mount an un-encryptable or wiped partition on /dev/block/by-name/userdata at /data options: discard,noflush_merge,fsync
```

(7) "fs_mgr" 和 "e2fsck"

这里标志是在检查和 mount 文件系统，如果失败会有类似如下的 log: fs_mgr_mount_all returned an error fs_mgr_mount_all returned unexpected error 255

(8) "zygote"

init: [name:bootprof&]BOOTPROF: 30090.716917:INIT:zygote-start: 这里表示要启动 zygote，如果没有这一句表示 zygote 还没启动，则要检查之前的 initrc 执行情况，文件系统 mount 情况

(9) "SysServerInit"

```
[ 68.947613] nbl_vm_srv[2580]: [vmid:0][ 36.379110][T601083] system_server:
[name:bootprof&]BOOTPROF: 36379.108394:Android:SysServerInit_START // 启动 system
server
```

```
[ 77.160856] nbl_vm_srv[2580]: [vmid:0][ 45.560415][T201083] system_server:
[name:bootprof&]BOOTPROF: 45560.411492:Android:SysServerInit_END
```

如果没有上述关键字，需要查看具体日志哪里发生了 crash。

(10) "system_server"

这里表示 system_server 有起来

```
(11) "init: [name:bootprof&]BOOTPROF: 48057.199422: OFF (KO:236)"
```

表示开机流程正常结束，进入安卓

3、抓 logcat 的情况:

```
[21.779899][T700289] init: [name:bootprof&]BOOTPROF: 21779.897282:INIT:zygote-start
zygote 起来后，android 系统起不来，可以抓 logcat
```

四、其他方式

1、adb shell ps -A | grep launcher launcher 进程是否启动

2、adb shell ps -A | grep zygote zygote 是否启动

3、adb shell mount, 正常示例如下

```
auto8676p1_64_bsp_vm:/ $ mount|grep "/data type f2fs"
/dev/block/dm-51 on /data type f2fs (rw, lazytime, seclabel, nosuid, nodev, noatime, background_gc=on, nogs_merge, discard, discard_unit=block, user_xattr, inline_xatt
r, acl, inline_data, inline_dentry, noflush_merge, barrier, extent_cache, mode=adaptive, active_logs=6, reserve_root=134217, resuid=0, resgid=1065, inlinecrypt, alloc_mo
de=default, checkpoint_merge, fsync_mode=nobarrier, memory=normal)
auto8676p1_64_bsp_vm:/ $ mount|grep "/system"
/dev/block/dm-9 on /system_ext type erofs (ro, seclabel, relatime, user_xattr, acl, cache_strategy=readaround)
/dev/block/dm-14 on /system_dlkm type erofs (ro, seclabel, relatime, user_xattr, acl, cache_strategy=readaround)
```

4、adb shell df -h

```

auto8676p1_64_bsp_vm:/ $ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/block/dm-8 598M 598M   0 100% /
tmpfs            4.3G 2.8M 4.3G   1% /dev
tmpfs            4.3G   0 4.3G   0% /mnt
/dev/block/dm-9 389M 389M   0 100% /system_ext
/dev/block/dm-10 842M 842M   0 100% /vendor
/dev/block/dm-11 146M 146M   0 100% /product
/dev/block/dm-12 14M 14M   0 100% /vendor_dkkm
/dev/block/dm-13 56K 56K   0 100% /odm_dkkm
tmpfs            4.3G 16K 4.3G   1% /apex
/dev/block/dm-51 60G 1.3G 58G   3% /data

```

- 3、和 4、都可以查看当前分区 mount 的状态
- 5、查看开机阶段 `adb shell cat /proc/bootprof`
- 6、另外可借助回读 `expdb` 分区，抓一些 `aee_exp` 信息协助分析。